

Online Exam Integrity Monitoring System

Progress Presentation

Dinis Contreiras (A50496) Miguel Sanches (A49337)

Supervisor: Prof. Jorge Martins



BSc in Computer and Software Engineering

April 2026

Index

Context and Requirements

Proposed Solution

System Architecture

Implementation Progress

Demonstration

Project Plan

Context

The widespread adoption of digital platforms in academic settings has transformed the way exams are performed, offering significant advantages when it comes to automation, accessibility, and even administrative efficiency for both educators and students.

This transition comes with associated costs and new challenges, particularly the preservation of academic integrity. When exams move from the paper world to laptops and computers, even with educators physically present in the room, the screen essentially becomes a **private space for the student**.

This creates a meaningful gap where the professor is present but effectively blind to what is happening on each student's machine.

Functional Requirements

- ▶ Professors can create and manage exams, start sessions and generate a session code for students to join.
- ▶ Students authenticate with their institutional email and access the exam through a web application.
- ▶ The daemon detects suspicious behavior including window focus loss, forbidden processes, clipboard usage and network changes.
- ▶ The daemon actively blocks clipboard access and attempts to terminate forbidden processes.
- ▶ A heartbeat mechanism detects client crashes or forced termination.
- ▶ Events are relayed in real time to the professor console, displaying each student's status and a timeline of occurrences.
- ▶ Session events are persistently logged on the server for later review.

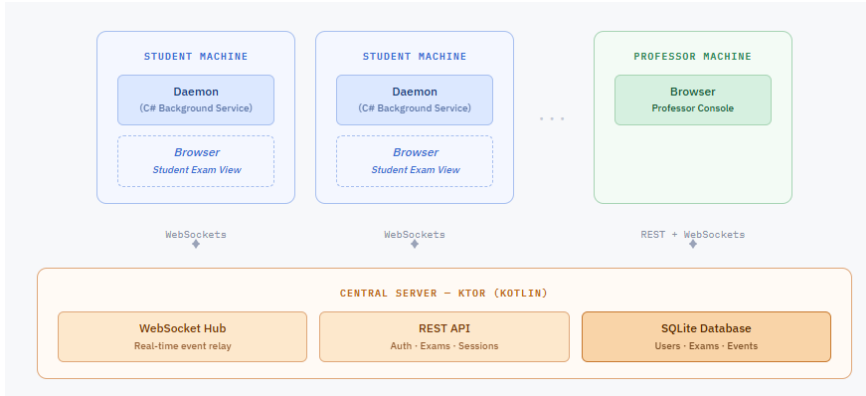
Solution

Our solution consists of two main components:

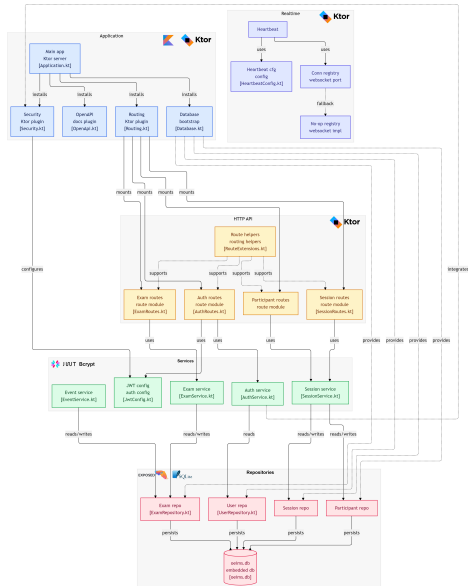
- ▶ A **lightweight daemon service** that runs silently on the student's machine during the exam, monitoring and actively mitigating suspicious behavior - detecting focus loss, forbidden processes, clipboard usage, and network changes.
- ▶ A **professor console** connected through a central server, receiving real-time alerts. A heartbeat mechanism ensures the professor is notified if a client stops running.

The exam itself is served through a **web application** where students authenticate with their institutional email and professors manage the full exam life cycle.

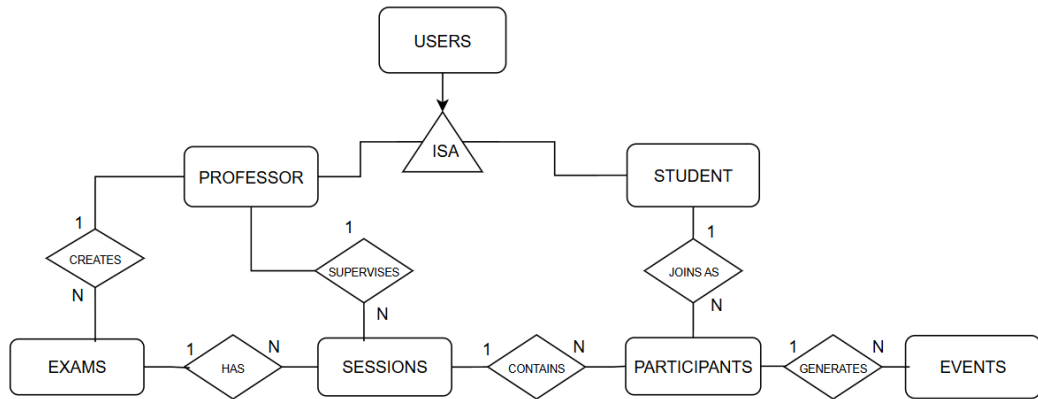
System Architecture



Block Diagram - Server



Data Model



Progress - Server

HTTP API Layer

- ✓ Registration and login endpoints
- ✓ JWT-based authentication
- ✓ Exam management endpoints
- ✓ Session lifecycle endpoints
- ✓ Student join endpoint
- ✓ Event querying endpoints

Testing Layer

- ✓ Repository layer tests
- ✓ Service layer tests
- ✓ In-memory test setup
- ✓ HTTP API layer tests

Realtime Layer

- ✓ Heartbeat endpoint
- ✓ Connection tracking
- ☐ Heartbeat timeout sweeper
- ☐ Daemon WebSocket channel
- ☐ Console WebSocket channel

Services Layer

- ✓ Authentication service
- ✓ Role-based authorization
- ✓ Exam service
- ✓ Session service
- ✓ Join code validation
- ✓ Event service

Repositories Layer

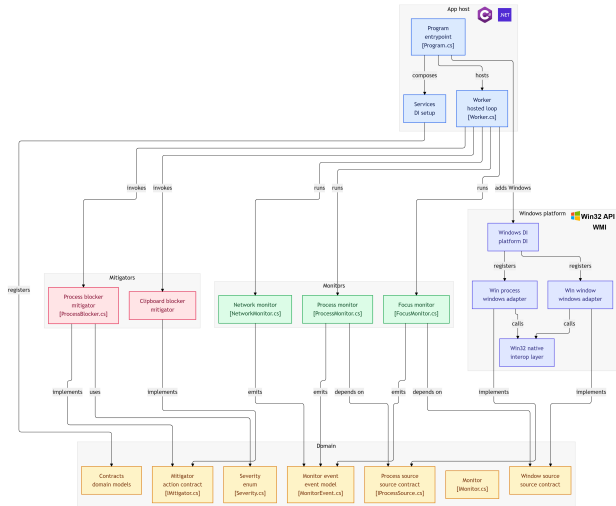
- ✓ User data access
- ✓ Exam data access
- ✓ Session data access
- ✓ Participant data access
- ✓ Event storage

REST API Documentation

Authentication Register a new account or log in to obtain a JWT token. ^		
POST	/auth/register	Register
POST	/auth/login	Login
Exams Create and query exams. Professor role required. ^		
POST	/exams	Create exam
GET	/exams	List exams
GET	/exams/{id}	Get exam
Participants Participant heartbeat. Student role required. ^		
POST	/participants/{id}/heartbeat	Heartbeat
Sessions Manage the exam session lifecycle – create, start, end, join. ^		
POST	/sessions	Create session
POST	/sessions/join	Join session
GET	/sessions/{id}	Get session
POST	/sessions/{id}/start	Start session
POST	/sessions/{id}/end	End session
GET	/sessions/{id}/participants	List participants
GET	/sessions/{id}/events	List session events

API reference available at `/docs` which is served directly by the server.

Block Diagram - Daemon



Progress - Daemon

Monitoring Layer

- ✓ Network state detection
- ✓ Process monitoring
- ✓ Focus tracking
- ☐ Windows focus abstraction

Mitigation Layer

- ✓ Process blocking
- ✓ Clipboard blocking

Testing Layer

- ☐ Monitor unit tests
- ☐ Worker integration tests
- ☐ Suspicious behavior simulation tests

Contracts Layer

- ✓ Shared event model
- ✓ Standardized daemon events
- ☐ Detailed event payloads

Host Layer

- ✓ Background worker setup
- ✓ Service lifecycle handling
- ☐ HTTP heartbeat loop
- ☐ Event streaming to server with WebSockets

Demonstration - REST API Flow

The following sequence was tested live using the IntelliJ HTTP Client, with tokens and identifiers propagated automatically via environment variables.

1. Register and login as professor and obtain JWT
2. Create an exam (title, duration)
3. Create a session and server returns a 6-character code
4. Start the session
5. Register and login as student and obtain JWT
6. Student joins the session using the code
7. Student sends heartbeat and server responds with 204 No Content
8. Professor ends the session

All endpoints return the correct status codes, payloads and timestamps.

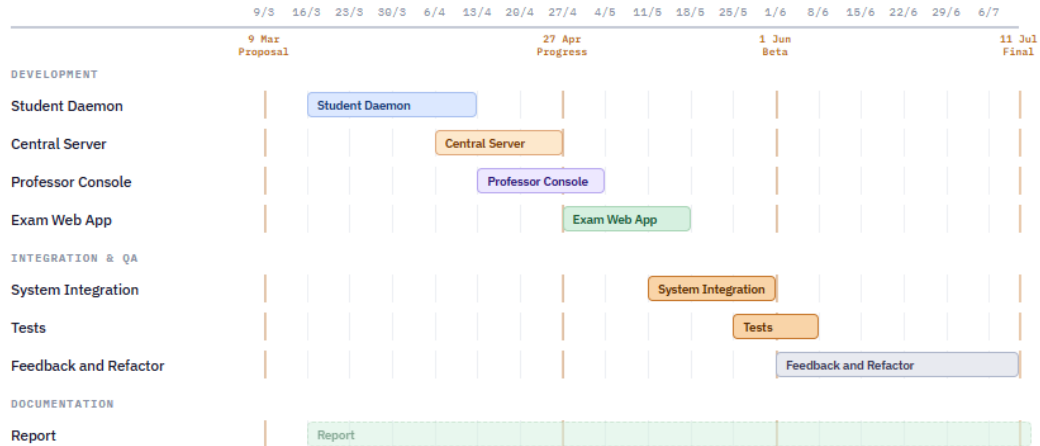
Demonstration - Daemon Flow

The following sequence demonstrates examples of suspicious actions detected and handled by the daemon.

1. Open a forbidden process and verify that the daemon terminates it in real time
2. Switch focus away from the exam window and verify that a focus-loss event is generated
3. Disable network and verify that a network event is detected

The demonstration validates real-time detection, mitigation, and event reporting.

Project Plan



Thank you

Questions & Comments

Online Exam Integrity Monitoring System

Dinis Contreiras (A50496) | Miguel Sanches (A49337)
Supervisor: Prof. Jorge Martins

BSc in Computer and Software Engineering – ISEL, 2026